实验 5：LLM、Context 与 RAG 实现

姓名：罗心月　　学号：24111302025　　班级：一班

1. 实验目的

- 编写并运行模型调用代码，掌握输入组织、结构化输出校验和调用失败处理。
- 完成资料切分、检索、上下文组织、回答生成与来源引用，理解 RAG 各环节的作用。
- 使用固定问题集对比直接调用模型与使用 RAG 的回答，分析效果与局限。

2. 实验内容

2.1 LLM 调用与输出格式

- 选择一个知识问答任务，说明用户问题、输入内容和结果用途；结合个人软件时，注明对应个人软件的用户故事。
- 编写并运行云端模型调用代码，记录模型名称、调用方式、配置和主要参数，展示完整输入与返回结果，并测试至少一种连接失败、接口不可用或超时情况。
- 运行至少 3 组输入，包含正常问题、缺少必要信息的问题和超出任务范围的问题，记录期望与实际结果。缺少信息时提示补充或追问，超出范围时按任务要求提示无法处理。

2.2 Prompt 与上下文组织

- 编写可重复使用的 Prompt 模板，区分任务要求和用户问题，明确回答格式及信息不足时的处理。完成 RAG 后加入检索资料与来源，多轮对话按需加入历史消息，并展示一份实际发送给模型的完整输入。
- 根据测试结果修改至少一次 Prompt，记录原版本、修改原因、修改内容和结果变化；结论限于本次样例，未改善或出现退步也应记录。
- 检查输入中是否存在无关或重复信息，按需精简并记录检查结果；有删减时比较长度与回答变化。选择和组织模型输入信息称为上下文工程；长度可用字符数或 Token 数。

2.3 知识库与切分策略

- 根据个人软件选择少量、来源明确的资料作为知识库；记录数量、格式、使用权限和隐私处理方式，清理乱码、重复及无关内容，不得向外部模型发送敏感数据。
- 按段落、标题或长度切分文本，说明片段大小、相邻片段是否重叠以及资料更新方法。
- 抽查至少 3 个片段，展示内容与元数据，检查关键语句是否完整、来源是否对应原资料。发现问题时调整切分方式并重新检查。

2.4 检索与 RAG 最小链路

- 实现关键词检索、向量检索或二者结合的方法，并说明选择理由。关键词检索按词语匹配；使用向量检索时，用同一嵌入模型分别将资料片段和用户问题转为数值向量，再比较相似程度，记录模型名称与比较方法。
- 展示一次查询、按相关性排序的前 k 个片段（Top-k）及排序依据，说明 k 的取值。返回片段不代表资料相关，应说明判断方法，如关键词匹配规则、经样例检查的相似度阈值或模型判断；无结果或资料不相关时，返回资料不足提示。
- 将检索片段与来源编号加入上下文，生成带出处的回答，检查引用是否对应原片段、是否支持回答；资料不足时明确提示，不编造答案或引用。
- 完成“问题输入—检索—上下文组织—模型调用—回答返回”的流程，保存检索片段、实际上下文和最终回答。

2.5 对比测试与质量分析

- 建立 5—8 个固定问题及判断规则，包含单片段可回答、多片段综合，以及资料无答案或含义不清的情况，写明处理方式，供后续实验重复使用。
- 两组使用相同问题、模型、主要参数和基本输出格式：直接模型组不提供检索资料，RAG 组增加资料及依据资料回答、引用来源的要求。说明两组提示词的差异；各次测试使用独立上下文，避免前一次结果影响后一次测试。
- 用统一表格记录两组回答和人工判断，比较正确性、要点遗漏和无依据内容，另检查 RAG 组的引用可靠性。资料无答案时，区分模型凭已有知识回答与资料能够支持回答。
- 记录从输入到返回结果的耗时，以及可获取的 Token 数或调用次数；收费接口补充实际或估算费用。

2.6 模块调用与后续使用

- 将检索和回答生成整理为可调用的模块，说明依赖、调用方式、输入、输出及失败返回，并提供知识库准备方法。
- 提供页面、接口或命令行入口，演示从输入问题到显示回答与出处的完整过程。
- 说明个人软件是否继续使用 RAG 及理由，需要时指出拟接入的功能；列出当前局限和下一步改进计划。

3. 操作记录

截图、代码和日志等均编号并配文字说明；数据来自 2026 年 10 月 26 日在本机的实际重跑，截图中的模型服务、检索片段、回答、Token 和费用均由程序输出。个别长输出用 `fold -s -w 88` 折行以便在终端内完整显示，折行不改变文字内容。

3.1 任务与模型调用

问答任务是“StudyPilot 资料问答”：用户用自然语言问项目或课程学习相关的问题，系统从本人编写的项目文档里检索资料，给出带出处的回答。结果用途是复习和查资料，不直接写入学习计划。对应的用户故事是实验 2 里的“作为本科生，我希望把课程要求转换成可修改的学习计划，以便按时完成任务并了解自己的薄弱知识点”，本实验补充的是其中的查资料环节。

模型调用代码在 `backend/app/service/llm_client.py`，用 httpx 调用 DeepSeek 的 Chat Completions 接口，密钥只从环境变量 `DEEPSEEK_API_KEY` 读取，不写进代码和日志。请求参数使用 `deepseek-chat`，实际返回的 `model` 字段以接口响应为准。

表 5-1: 模型调用参数

项目	设置
模型名称	请求参数 `deepseek-chat`
调用方式	HTTP POST，`https://api.deepseek.com/chat/completions`
主要参数	temperature 0.2（任务解析用 0）、max_tokens 700—900、stream=false、超时 60 秒
结构化输出	RAG 回答开启 `response_format=json_object`，再用 Pydantic 校验字段
密钥管理	环境变量 `DEEPSEEK_API_KEY`，日志和截图只显示前 6 位与后 3 位

```
$ source tmp/env.sh && cd backend && python3 -m app.tools.ask_cli "StudyPilot 里知识点掌握度是怎么更
新的？" 2>/dev/null | fold -s -w 88
模型： deepseek-chat    密钥： 已配置 sk-66e***604
问题： StudyPilot 里知识点掌握度是怎么更新的？
模式： rag    Prompt 版本： v3    Top-k： 3
--------------------------------------------------------------------------------
检索到 3 个片段：
  [1] 分数 32.782  《 StudyPilot 数据模型说明》  StudyPilot 数据模型说明 / 表结构 /
knowledge_points 知识点表 （ S5-004）
  [2] 分数 26.75   《 StudyPilot 项目说明》  StudyPilot 课程实验报告 /
资料问答 （ RAG） （ S4-004）
  [3] 分数 25.332  《 StudyPilot 项目说明》  StudyPilot 课程实验报告 /
资料问答 （ RAG） （ S4-005）
--------------------------------------------------------------------------------
回答：
结论： 知识点掌握度由用户填写或确认后更新， 取值为 0–100 的整数。 [1]
依据： knowledge_points 表的 mastery 字段定义为“掌握度， 0–100
的整数， 由用户填写或确认后更新”， 并记录 updated_at 最后更新时间。 [1]
资料未说明具体的更新操作流程或界面步骤。
--------------------------------------------------------------------------------
是否依据资料回答： 是
引用：
  [1] StudyPilot 数据模型说明 / StudyPilot 数据模型说明 / 表结构 / knowledge_points
知识点表， 词语重合度 0.512， 对应
耗时： 1149 ms    输入 670 tokens， 输出 95 tokens， 合计 765 tokens    费用约 0.0021 元
```

图 5-1: 一次完整调用：检索片段、回答与出处

```
$ source tmp/env.sh && cd backend && python3 -m app.tools.ask_cli "StudyPilot 里知识点掌握度是怎么更
新的？" 2>/dev/null | fold -s -w 88
模型： deepseek-chat    密钥： 已配置 sk-66e***604
问题： StudyPilot 里知识点掌握度是怎么更新的？
模式： rag    Prompt 版本： v3    Top-k： 3
--------------------------------------------------------------------------------
检索到 3 个片段：
  [1] 分数 32.782  《 StudyPilot 数据模型说明》  StudyPilot 数据模型说明 / 表结构 /
knowledge_points 知识点表 （ S5-004）
  [2] 分数 26.75   《 StudyPilot 项目说明》  StudyPilot 课程实验报告 /
资料问答 （ RAG） （ S4-004）
  [3] 分数 25.332  《 StudyPilot 项目说明》  StudyPilot 课程实验报告 /
资料问答 （ RAG） （ S4-005）
--------------------------------------------------------------------------------
回答：
结论： 知识点掌握度由用户填写或确认后更新， 取值为 0–100 的整数。 [1]
依据： knowledge_points 表的 mastery 字段定义为“掌握度， 0–100
的整数， 由用户填写或确认后更新”， 并记录 updated_at 最后更新时间。 [1]
资料未说明具体的更新操作流程或界面步骤。
--------------------------------------------------------------------------------
是否依据资料回答： 是
引用：
  [1] StudyPilot 数据模型说明 / StudyPilot 数据模型说明 / 表结构 / knowledge_points
知识点表， 词语重合度 0.512， 对应
耗时： 1149 ms    输入 670 tokens， 输出 95 tokens， 合计 765 tokens    费用约 0.0021 元

$
```

图 5-2: 调用返回：引用核对、耗时、Token 与费用

调用失败处理按类型区分，接口返回统一结构的错误信息，程序不会中断。

表 5-2: 三种调用失败情况的实测结果

情况	错误码	提示信息	接口返回
API Key 无效	LLM_AUTH_FAILED	模型服务鉴权失败（401），请检查 API Key	HTTP 502
响应超时（超时设为 1 毫秒）	LLM_TIMEOUT	响应超时（ConnectTimeout），请稍后重试	HTTP 504
服务地址不可达	LLM_CONNECTION_ERROR	无法连接模型服务，请检查网络或地址	HTTP 503



图 5-3: 调用失败实测：鉴权失败、超时、地址不可达

结构化输出用“把自然语言学习要求解析成任务”来验证：字段为 title、minutes、date、point，日期用正则约束为 YYYY-MM-DD。正常输入一次通过，本次耗时 10479 毫秒、153 tokens（这次调用遇到模型服务延迟波动，日志中也能看到同一次调用的长耗时）；故意换成模型无法满足的结构（要求 priority、owner 字段）时，第一次校验失败，程序把校验错误回传给模型并要求只返回合法 JSON，第二次通过，重试样例耗时 2433 毫秒、386 tokens。

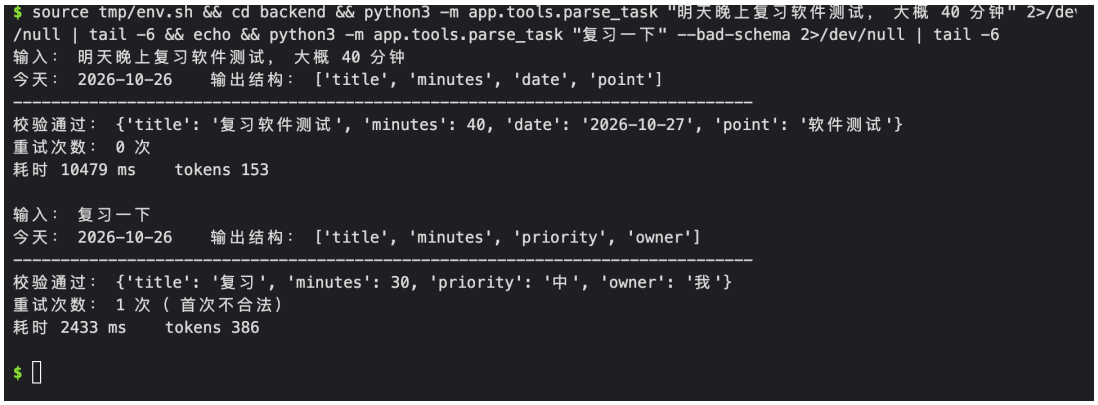


图 5-4: 结构化输出：正常解析与校验失败后重试

三组输入的期望与实际结果如下。第一组是正常问题；第二组缺少必要参数，期望先追问而不是直接给设计说明；第三组超出问答范围，期望检索无结果时返回资料不足并且不调用模型。

表 5-3: 三组输入的期望与实际结果

类型	输入	期望	实际结果
正常问题	StudyPilot 里知识点掌握度是怎么更新的?	依据资料说明由用户填写或确认后更新，并给出来源	回答“由用户填写或确认后更新，取值 0—100 的整数，并记录最后更新时间”，引用 [1]；同时说明资料没有写自动计算机制
缺少必要信息	帮我设置考试目标	指出缺少课程、考试日期、目标分数和每天可用时间，请用户补充，不直接给设计说明	先说明无法直接设置，列出需要补充的课程、考试日期、目标分数和可用时间；但仍带出了 exam_goals 字段和接口，回答偏长，本次 sufficient 仍为 true
超出任务范围	帮我写一首关于春天的诗	检索无结果时返回资料不足，不调用模型	返回“资料不足：知识库中没有找到与该问题相关的内容”，未调用模型，耗时 0 ms、0 tokens

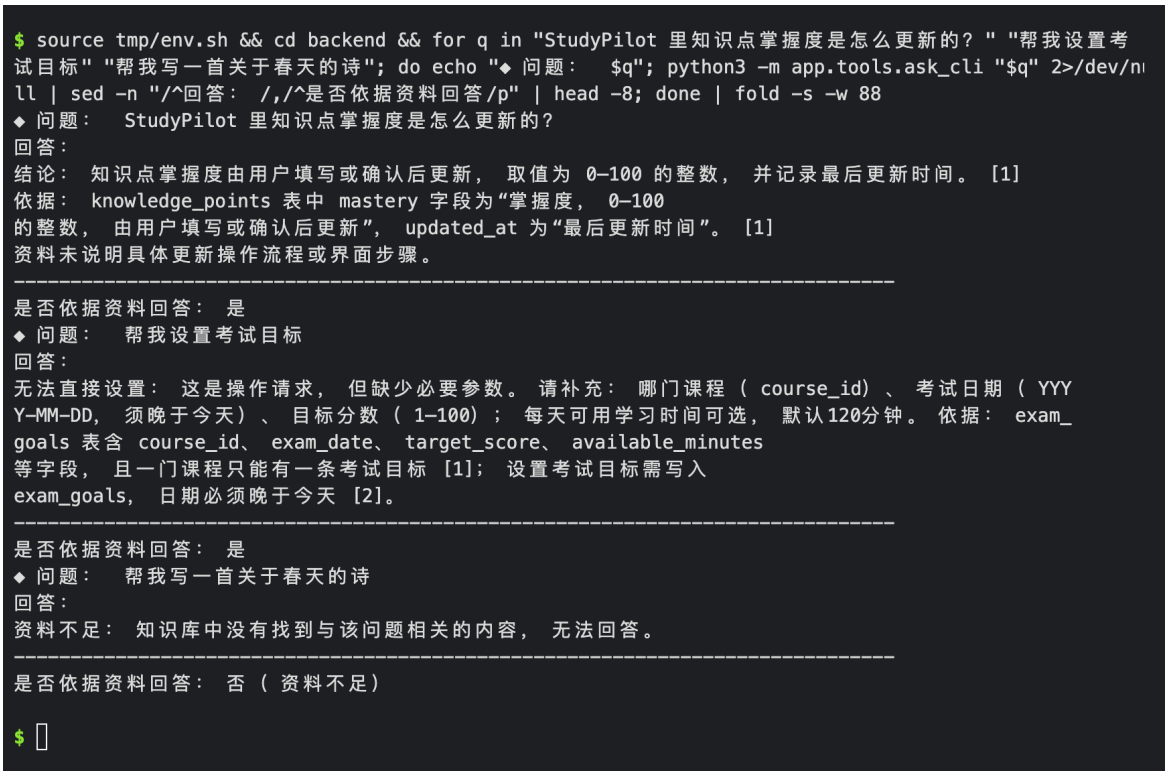


图 5-5: 三组输入的实际反馈

3.2 Prompt 与上下文组织

Prompt 模板放在 `backend/app/service/prompts.py`，任务要求与用户问题分开：模板里写清回答范围、长度、引用格式、信息不足和超范围时的处理，最后才是资料和用户问题。版本变化见表 5-4，实际发送给模型的完整输入见图 5-8 和图 5-9。

表 5-4: Prompt 修改记录

版本	原版本问题	修改内容	结果变化
v1	RAG 只写“请根据资料回答问题”	无格式要求，也不要求结构化输出	回答明显更长，使用 Markdown 加粗和列表；引用编号 [2][3][1] 由模型自行写出，格式不受约束、未做校验
v2	回答长度和引用不规范	增加只依据资料、250 字以内、句末标注 [1]、资料不足时置 sufficient=false、只输出 JSON	回答压缩到约一半长度，引用编号可程序提取并核对；资料不足时能明确说明
v3	缺少参数的操作请求（如“帮我设置考试目标”）被当成普通问题回答	增加一条：操作请求缺少参数时先列出需要补充的信息	“帮我设置考试目标”改为先问课程、考试日期、目标分数和每天可用时间；但回答中仍带出字段名和接口，追问不够简洁

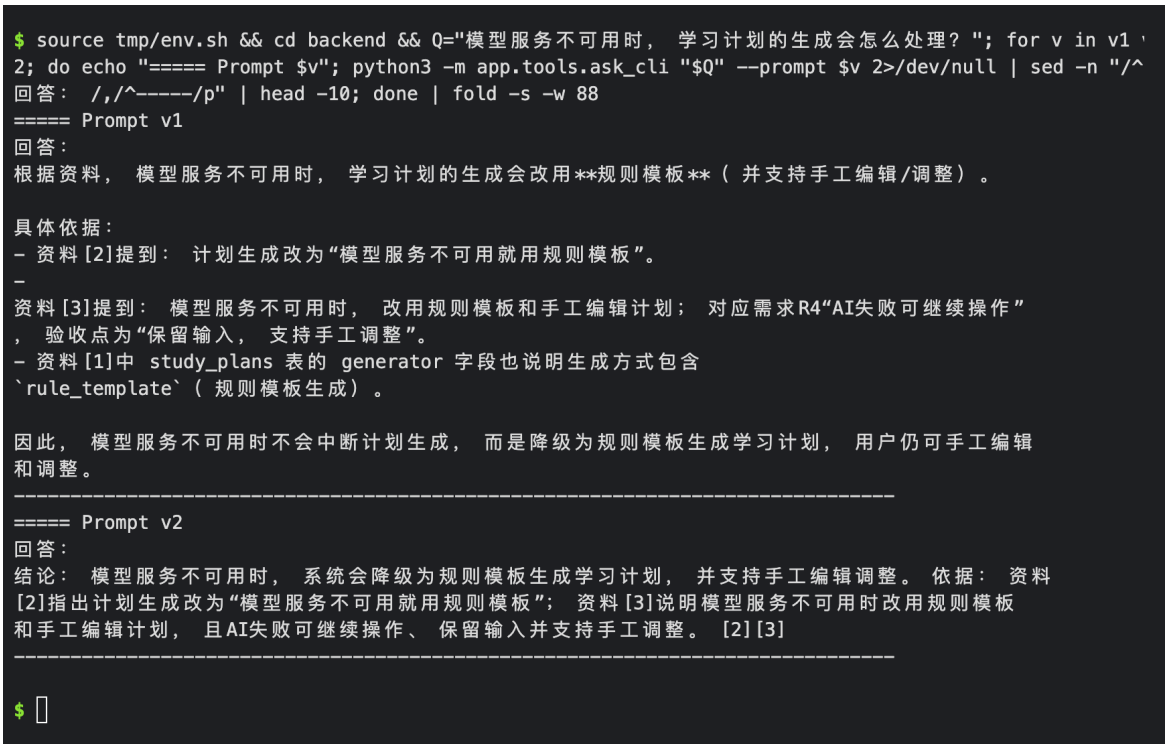


图 5-6: Prompt v1（上）与 v2（下）对同一问题的回答对比

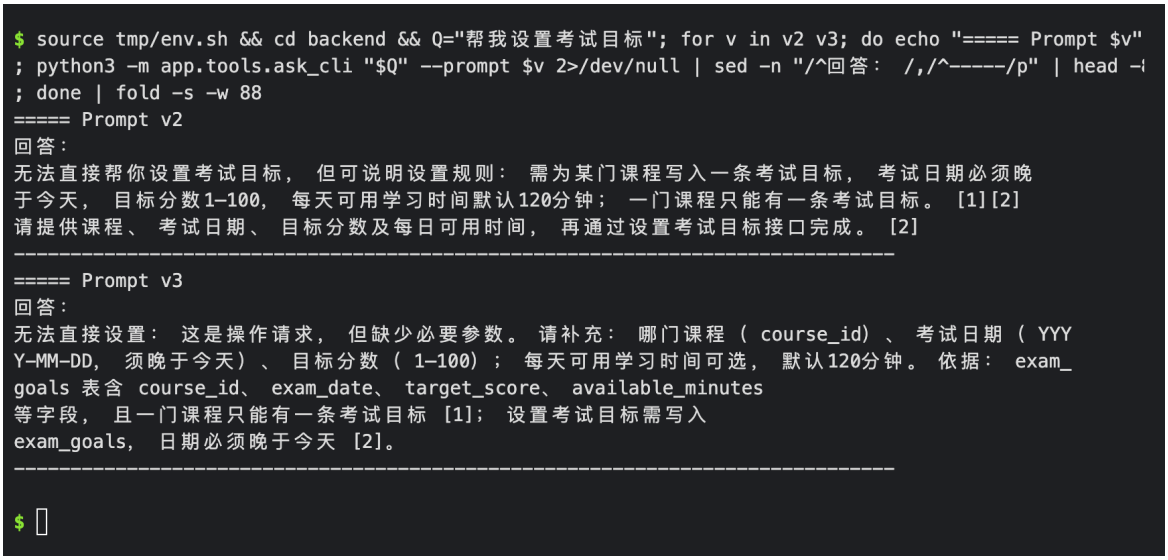


图 5-7: Prompt v2（上）与 v3（下）对“帮我设置考试目标”的回答对比

实际发送给模型的输入只有一条 user 消息：先是任务要求，再是带编号的检索资料（每条含来源标题、章节和正文），最后是用户问题；本次没有多轮历史，因此没有加入历史消息。

```
$ source tmp/env.sh && cd backend && python3 -m app.tools.ask_cli "StudyPilot 的总体架构分为哪几层?
" --show-context --only-context 2>/dev/null | fold -s -w 88
模型： deepseek-chat    密钥： 已配置 sk-66e***604
问题： StudyPilot 的总体架构分为哪几层?
模式： rag    Prompt 版本： v3    Top-k： 3
--------------------------------------------------------------------------------
检索到 3 个片段：
  [1] 分数 21.704  《 StudyPilot 项目说明》  StudyPilot 课程实验报告 /
资料问答 （ RAG） （ S4-005）
  [2] 分数 18.889  《 实验3 软件架构与界面设计》  实验3： 软件架构与界面设计 / 3. 操作记录
/ 3.2 架构与核心流程设计 （ S2-009）
  [3] 分数 11.799  《 实验3 软件架构与界面设计》  实验3： 软件架构与界面设计 / 2. 实验内容
/ 2.3 软件架构与模块设计 （ S2-004）
--------------------------------------------------------------------------------
实际发送给模型的输入 （ messages） ：
[
  {
    "role": "user",
    "content": "你是
StudyPilot （ 学习任务规划与知识追踪系统） 的资料问答助手。 \n任务要求： \n1.
只依据下面给出的资料回答， 不要使用资料以外的知识； \n2. 回答不超过 250
字， 先给结论， 再给依据； \n3. 引用资料时在句末标注编号， 例如 [1]， 编号必须来自资料； \n4.
如果用户提出的是操作请求但没有给出必要参数 （ 例如只说“帮我设置考试目标”） ， 请列出需要补充
的信息 （ 哪门课程、 考试日期、 目标分数等） ， 不要直接给设计说明； \n5.
如果资料不能回答这个问题， 把 sufficient 设为 false， 并在 answer
中说明“资料不足”及缺少什么信息； \n6. 如果问题与 StudyPilot
项目或软件工程课程学习无关， 直接说明无法处理； \n7. 只输出 JSON， 格式： {\"answer\":
\"回答内容\", \"citations\": [1, 2], \"sufficient\": true}。 \n\n资料： \n[1]
来源：《 StudyPilot 项目说明》  章节： StudyPilot 课程实验报告 /
资料问答 （ RAG） \np.tools.ask_cli \"知识点掌握度怎么更新? \n\"     # 命令行问答\npython3
-m app.tools.ask_cli \"计划怎么生成? \n\" --show-retrieval --only-context\npython3 -m
app.tools.inspect_chunks --source S2 --limit 3   # 片段抽查\npython3 -m
```

图 5-8: 实际发送给模型的输入：任务要求与资料编号

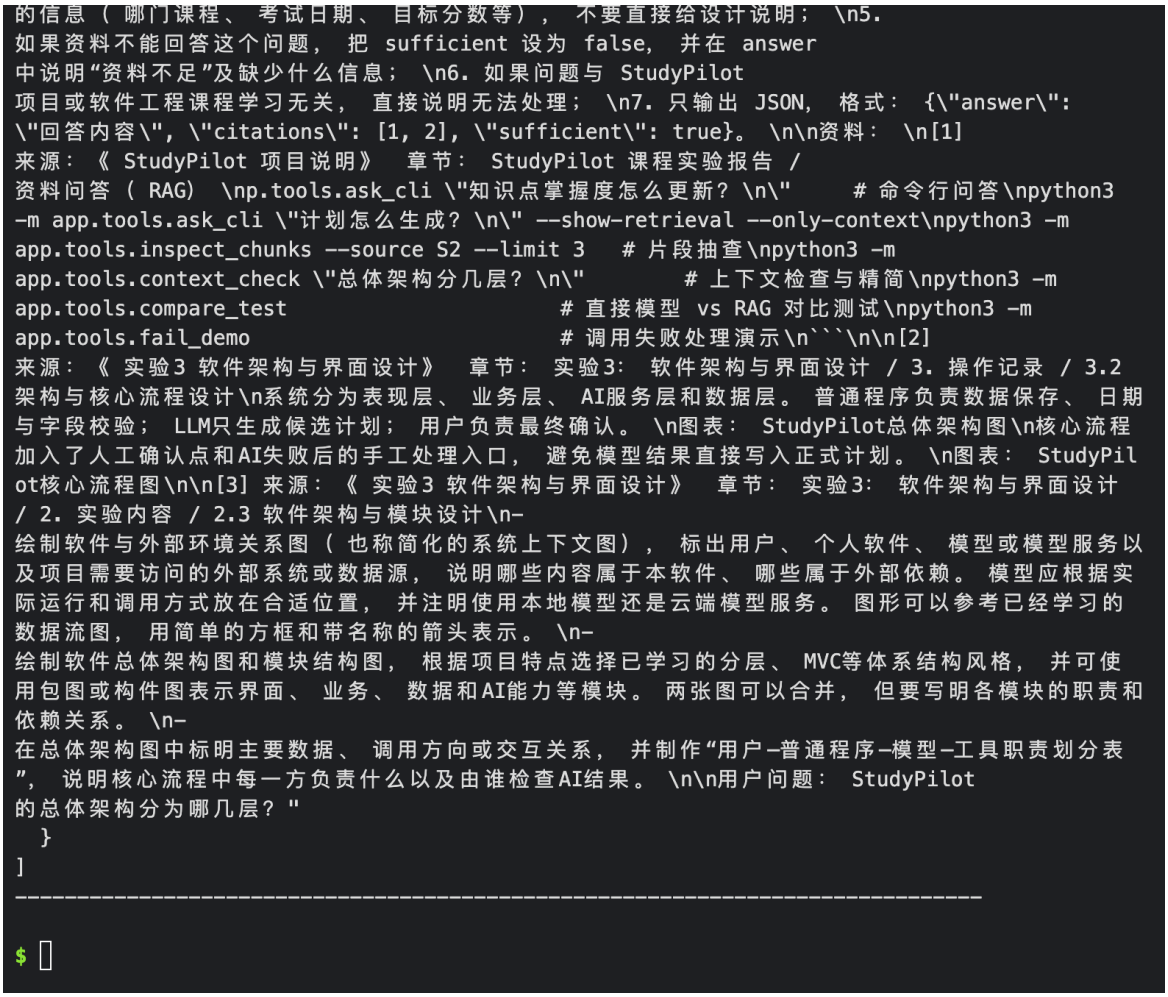


图 5-9: 实际输入中的资料内容与用户问题

上下文检查用架构问题做样例，为了看清边缘片段，把 Top-k 调到 6、相关性阈值调到 3。检查发现两类问题：一是检索排在后面的片段与问题无关（第 5 条是“编译实验报告”的 LaTeX 命令，第 6 条是实验目的，都不能回答架构分层）；二是重复内容：S4-005 在切分时与前一片段有 80 字重叠；另外 README 源文件里 xelatex 编译命令重复出现，两个 README 片段把同一句“xelatex 实验 3-软件架构与界面设计.tex”带进上下文，重复句子共 25 字。按“分数不低于最高分 60%”精简后，上下文从 1577 字降到 657 字（减少 58%），prompt tokens 从 1080 降到 541，回答仍给出四层名称并引用 [2]，引用没有变化。

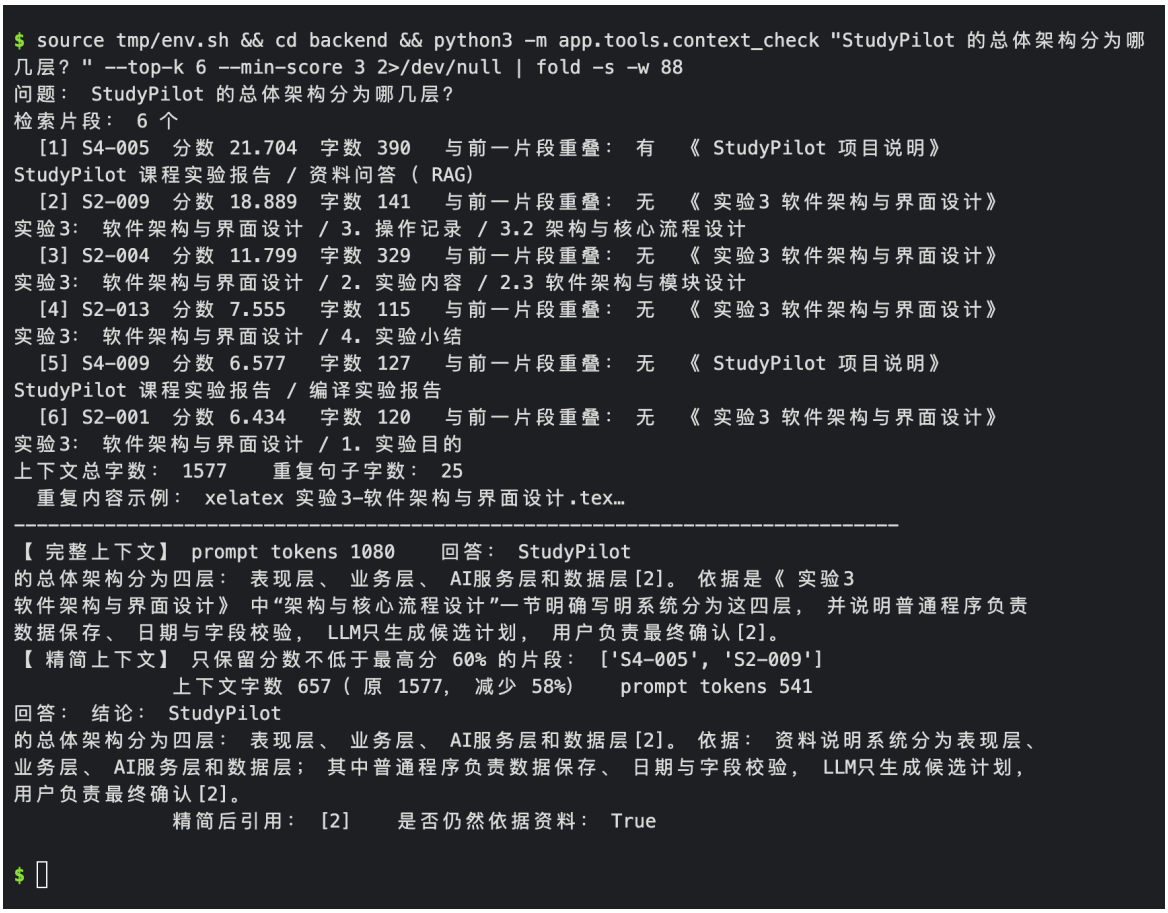


图 5-10: 上下文检查：无关片段、重复内容与精简前后对比

需要说明的是，按分数精简并不能去掉所有噪声：3.1 的调用中，检索前 3 名有两名来自《StudyPilot 项目说明》的“资料问答（RAG）”小节（分数 26.75、25.33），它们只是在讲命令用法，并不是数据模型说明；因为分数高，60% 阈值仍会保留。这说明关键词检索的分数只能部分反映相关性，后续需要更好的排序或人工可调的过滤规则。

3.3 知识库与切分策略

知识库共 5 篇资料，全部是本人编写的项目文档，来源明确、可以公开，不包含他人隐私、密钥和真实用户数据。资料清单放在 `backend/kb/sources.json`，每条记录格式、使用权限和隐私处理方式。当前构建共 77 个片段。

表 5-5: 知识库资料清单

编号	资料	来源文件	格式与权限	字数	片段数
S1	实验 2 个人软件选题与需求分析	reports/experiment2/…tex	LaTeX；本人报告，可公开	3957	17
S2	实验 3 软件架构与界面设计	reports/experiment3/…tex	LaTeX；本人报告，可公开	3025	13
S3	实验 4 软件基础功能开发	reports/experiment4/…tex	LaTeX；本人报告，可公开	7179	29
S4	StudyPilot 项目说明	README.md	Markdown；本人项目文档	1882	10
S5	StudyPilot 数据模型说明	backend/kb/source/数据模型说明.md	Markdown；本人整理，可公开	1696	8

清理与切分：LaTeX 报告先转成纯文本，去掉导言区、图片和大部分排版命令，表格按行转成“单元格 | 单元格”，图只有标题时保留“图表：标题”一行；Markdown 去掉重复标题并保留正文。清理并不彻底：抽查发现 S4-003—S4-009 仍保留三段代码围栏，S3-014、S3-017 残留“p{5.8cm}X p{4.0cm}}”之类的表格列宽串，README 中 xelatex 命令重复两次；这些内容会作为噪声进入检索，已列入 3.6 的局限和下一步改进。然后按标题切成小节，小节内按段落合并到 420 字左右，同一小节被切开时保留上一片段末尾 80 字作为重叠。更新方法是修改源文件后重新执行 `python3 -m app.tools.build_kb`，片段编号会重新生成；本次重建前是 74 个片段，补充 README 的资料问答说明后变为 77 个，说明资料更新后必须重建索引。

```
构建时间：  2026-10-26 09:58:44
切分参数：  片段 420 字，  相邻重叠 80 字
片段总数：  77

编号  资料                          字符数   片段数  更新时间
S1    实验2 个人软件选题与需求分析    3957      17  2026-09-20
S2    实验3 软件架构与界面设计        3025      13  2026-09-22
S3    实验4 软件基础功能开发          7179      29  2026-10-12
S4    StudyPilot 项目说明             1882      10  2026-10-16
S5    StudyPilot 数据模型说明         1696       8  2026-10-16

$
```

图 5-11: 知识库构建结果：5 篇资料、77 个片段

资料均为本人课程报告或项目文档，不含他人隐私、密钥或真实用户数据；README 中含本人姓名和学号，属于本人公开信息，S5 为表结构说明，不含真实数据。

抽查了 3 个片段：S2-009（架构分层，141 字，无重叠）、S5-004（知识点表字段，157 字，无重叠）、S3-019（非 AI 业务闭环的验收记录，381 字，与前一片段重叠）。检查结果：关键句子完整，“系统分为表现层、业务层、AI 服务层和数据层”没有在切分处断开；来源路径、章节和更新时间都能对应回原资料；带重叠的片段开头确实重复了上一片段的结尾，用于避免跨片段的问题漏检。

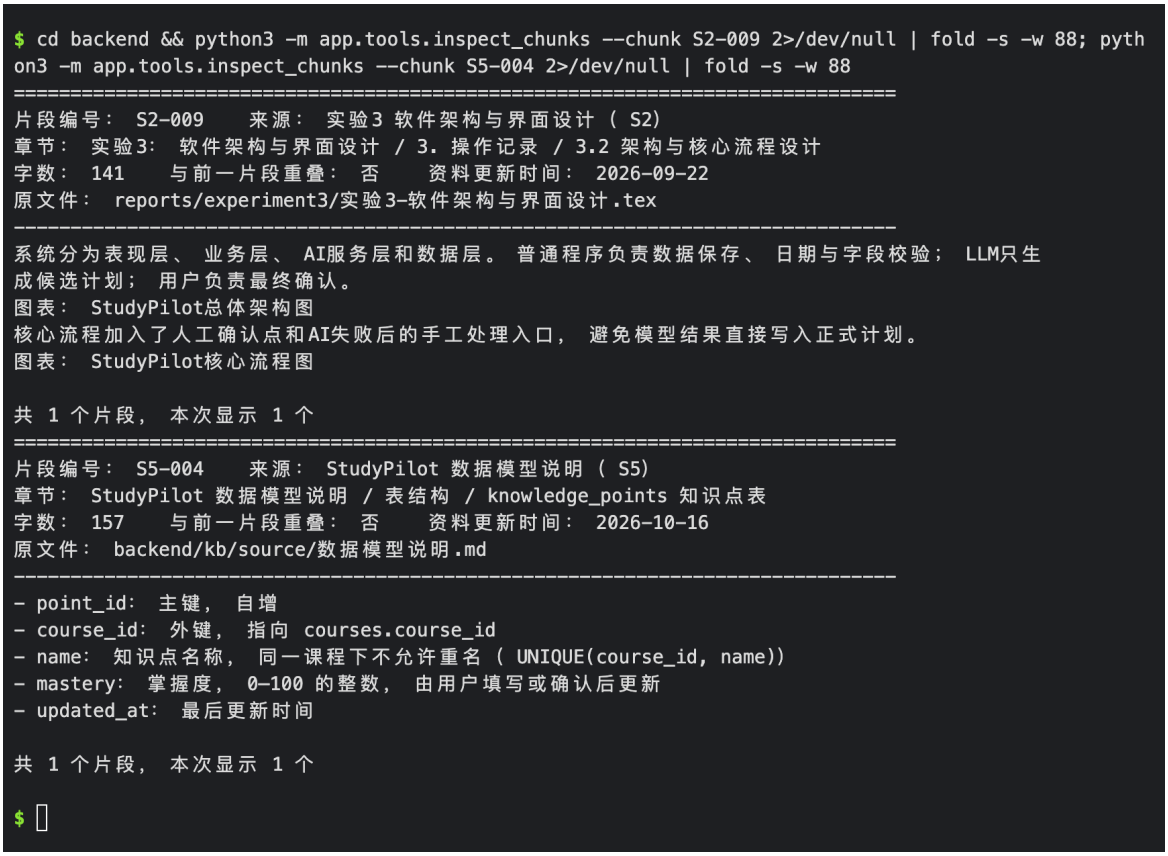


图 5-12: 片段抽查一：S2-009 与 S5-004 的内容与元数据

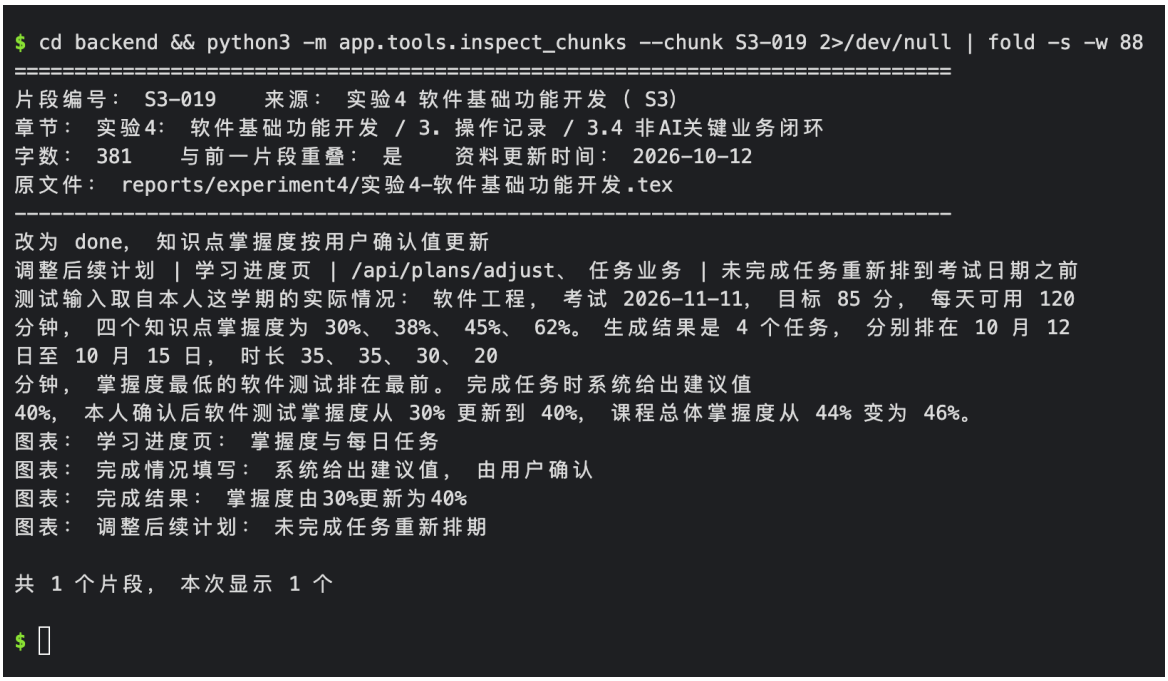


图 5-13: 片段抽查二：S3-019 的内容与重叠标记

3.4 检索与 RAG 最小链路

检索用关键词方法：中文按二字切分、英文按单词切分，用 BM25 排序（k1=1.5，b=0.75），并加入标题字段加权和同义词扩展。选择理由是没有配置嵌入模型，资料量只有几十个片段，关键词检索的排序依据可以逐项核对，不需要额外的模型服务。主要参数见表 5-6。

表 5-6: 切分与检索参数

项目	取值与说明
片段大小	420 字，相邻片段重叠 80 字
分词方式	中文二字组合 + 英文单词，去停用词
排序方法	BM25，标题字段权重 1.2
同义词扩展	权重 0.6，例如“数据库 → 表/字段/主键/外键”
相关性阈值	10 分，低于阈值视为没有相关资料
Top-k	默认 3，页面可调 1—8

阈值是抽查后定的：有答案的问题首位片段在 17.7—32.9 分之间（例如架构 23.0、掌握度 32.9、数据库关系 20.7）；话题相关但资料没有答案的问题（知识追踪算法）首位 12.6 分；与项目完全无关的问题检索为空。取 10 分可以把“有资料”和“完全无关”分开，

话题相关但资料不足的交给模型判断并把 sufficient 置为 false。

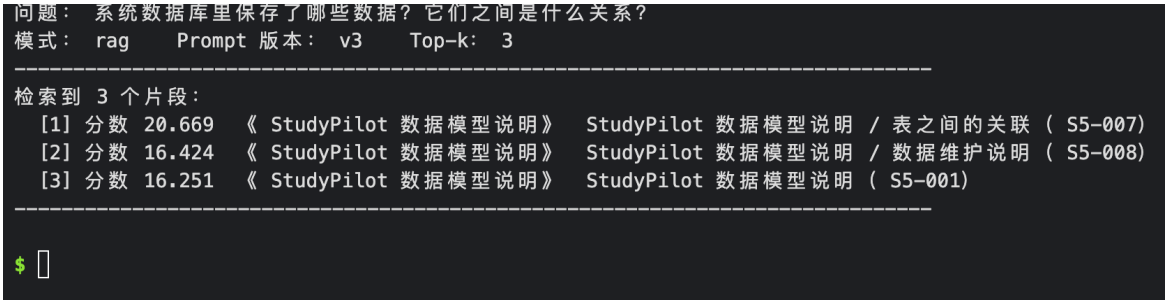


图 5-14: 一次查询的 Top-3 片段与 BM25 分数

样例检查发现过一次检索失败：问“系统数据库里保存了哪些数据？它们之间是什么关系？”时，前 3 个片段都是实验 3 的设计说明，没有检索到数据模型的内容，回答只能给出资料不足。原因是问题和资料用词不一致（问“数据库、数据”，资料里写的是“表、字段、主键”），而且正文里的通用词太多。加入标题加权和同义词扩展后，数据模型说明的“表之间的关联”片段排到第 1，回答也正确了。

表 5-7: 检索改进前后对比（同一个问题）

阶段	Top-3 片段（分数）	回答情况
改进前	S2-004(11.2)、S2-001(7.4)、S2-006(6.1)	资料不足，未回答
改进后	S5-007(20.7)、S5-008(16.4)、S5-001(16.3)	正确列出用户、课程、考试目标、知识点、计划、任务及关联，并给出引用

RAG 链路为：问题输入 → 检索 Top-k 片段 → 按“[编号] 来源/章节/正文”组织上下文 → 调用模型并校验 JSON → 检查引用编号并返回回答。引用核对会检查编号是否在检索结果范围内，并计算被引用片段与回答的词语重合度，低于 0.12 标记为需要人工复核，编号越界直接判为不可靠。3.5 的对比测试中 RAG 组共引用 13 处：12 处重合度不低于 0.12，1 处（Q6 的 [3]，S5-001）为 0.098，需要人工复核；逐条回看片段，发现它只是数据模型说明的来源元数据，不能直接支持表关系，属于弱引用。没有出现编号越界。

资料不足分两种处理：检索完全没有结果时（如“项目开发一共花了多少钱”）直接返回资料不足，不调用模型；检索到了片段但资料回答不了时（如“用的是哪种知识追踪算法”），把 sufficient 置为 false，并在回答里说明缺少什么信息。对比测试的完整记录保存在 `backend/tests/data/compare_result.json`，包含问题、检索片段、实际上下文、回答、引用检查、耗时和 Token。



图 5-15: 页面入口：RAG 回答、出处与检索片段



图 5-16: 资料不足时的回答：说明缺少什么信息，不编造

3.5 对比测试与质量分析

固定问题集放在 backend/tests/data/compare_questions.json，共 8 题：3 题单片段可回答（Q1、Q2、Q4）、3 题多片段综合（Q3、Q5、Q6）、2 题资料无答案（Q7、Q8）。判断规则写明每题需要覆盖的要点；资料无答案的两题，正确做法是说明资料不足，不能编造数字或算法名称。两组使用相同的问题、模型和 temperature（0.2），直接组使用 DIRECT_V3（不提供资料，200 字以内，缺少参数时追问，超范围时拒绝），RAG 组使用 RAG_V3（只依据资料、250 字以内、句末标注 [1]、资料不足时置 sufficient=false、只输出 JSON），RAG 组多出资料和引用要求；每次调用都是独立的单轮对话，不复用上一次上下文。

表 5-8: 对比测试逐题结果（Prompt v3，Top-k=3；耗时与 Tokens 两列为“直接/RAG”）

编号	类型	问题	要点 直接	要点 RAG	引用	耗时 ms	Tokens
Q1	单片段	总体架构分为哪几层	0/3	3/3	1	1239 / 1462 ms	214 / 905
Q2	单片段	掌握度自动修改还是用户确认	1/2	2/2	2	1252 / 1386 ms	195 / 841
Q3	多片段	模型不可用时计划怎么生成	0/2	2/2	3	1666 / 1374 ms	223 / 866
Q4	单片段	MVP 不实现哪些功能	0/2	2/2	2	1797 / 1564 ms	234 / 980
Q5	多片段	测试覆盖哪几类异常	0/2	2/2	2	1782 / 1371 ms	195 / 945
Q6	多片段	数据库保存哪些数据及关系	3/3	3/3	2	1733 / 1474 ms	235 / 709
Q7	资料无答案	用的是哪种知识追踪算法	无要求	无要求	1	1443 / 1242 ms	209 / 383
Q8	资料无答案	开发花了多少钱	无要求	无要求	0	1215 / 0 ms	182 / 0

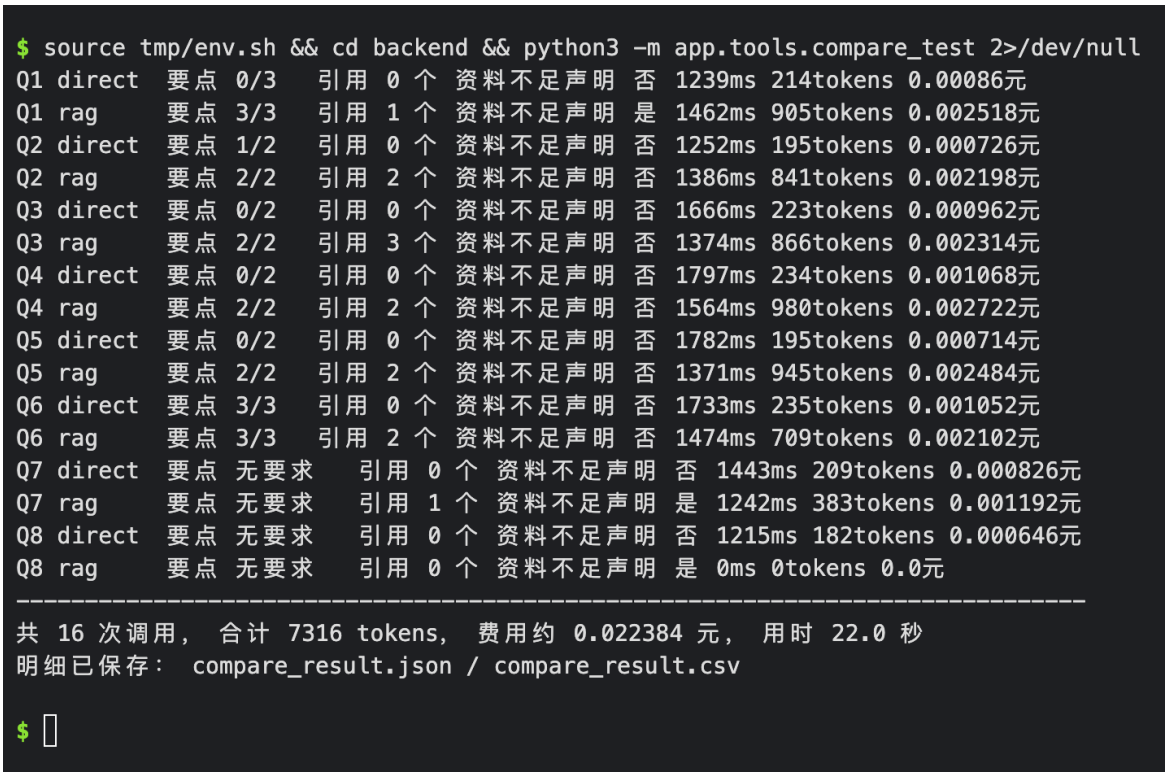


图 5-17: 对比测试运行结果：16 次测试（实际 15 次模型调用），7316 tokens，约 0.0224 元

表 5-9: 两组回答摘录与人工判断

编号	直接组回答摘录	人工判断	RAG 组回答摘录	人工判断
Q1	表示层、应用层、领域层、基础设施层	分层名称不符，0/3	表现层、业务层、AI 服务层、数据层；末尾说明职责资料不足	结论正确，3/3
Q2	系统按练习/测验自动更新，用户可确认	结论部分错，1/2	用户确认后更新，系统只给建议	正确，2/2
Q3	规则/模板生成，未提手工入口	漏要点，0/2	规则模板 + 手工编辑调整	正确，2/2
Q4	多用户、社交、推荐、移动端等	多为无依据内容，0/2	学习通、题库、社交、多端同步等	与资料一致，2/2
Q5	拒答，要求补充实验 4 文档	未回答，0/2	四类异常 + 两个具体例子	正确，2/2
Q6	用户/课程/任务/计划/知识点/掌握度记录；任务关联计划	关键词 3/3，关系有误	用户—课程—知识点/计划—任务，字段未列全	正确，3/3；1 处弱引用
Q7	列 BKT/DKT/IRT 候选，建议查文档	未编造，但未说资料不足	明确资料不足，缺算法说明	处理正确，1 处引用
Q8	以超出范围为由拒绝	未编造	检索无结果，资料不足，未调用模型	处理正确，无引用

本次共执行 16 次测试运行（Q8 的 RAG 组因检索无结果未调用模型，实际模型调用 15 次），合计 7316 tokens，按输入 2 元/百万、输出 8 元/百万估算约 0.0224 元，总用时 22.0 秒。直接组 8 次共 1687 tokens、约 0.00685 元；RAG 组 7 次调用共 5629 tokens、约 0.01553 元。RAG 的 tokens 约为直接组的 3.3 倍（输入 tokens 约 4.4 倍），费用约为

2.3 倍；耗时两组接近（直接组合计 12.1 秒、RAG 组合计 9.9 秒），本次网络波动较大，不能据此认为 RAG 一定更快或更慢。

人工检查回答内容，主要差别有四点。第一是正确性：直接模型在项目专属问题上给出了看似合理但和项目不符的内容。Q1 把架构说成“表示层、应用层、领域层、基础设施层”，项目实际是“表现层、业务层、AI 服务层、数据层”；Q2 说“系统根据练习/测验结果自动更新掌握度”，实际是用户填写或确认后更新；Q3 提到规则/模板但没有手工调整入口；Q4 编造了“多用户协作/社交、复杂推荐算法、移动端 App、第三方日历、知识图谱可视化、自动出题与批改”等 MVP 排除项；其中只有“社交、知识图谱”等少数与资料重合，其余在资料中没有依据。RAG 组的回答都能在原片段里找到对应句子。第二是资料无答案时的处理：直接模型 Q7 没有编造具体算法，而是列出 BKT、DKT、IRT 等候选并建议查看设计文档，Q8 以超出范围为由拒绝；RAG 组 Q7 明确说明资料不足并指出缺少算法说明，Q8 说明知识库没有费用信息且未调用模型，两题都没有编造。第三是引用可靠性：RAG 组 13 处引用中 12 处通过重合度阈值，1 处弱引用已人工标注，没有编号越界；但“能引用”不等于“引用足以支持结论”，Q6 的弱引用就是例子。第四是自动要点打分的局限：Q6 直接组按关键词判为 3/3，但它把掌握度说成“掌握度记录”、把任务说成关联计划，和实际数据模型有出入，说明关键词命中只能作为初筛，最终还要人工判断。

也有 RAG 答得不够好的地方：Q1 已经正确说出四层名称，却在末尾补“各层详细分工资料不足”，被程序标成“资料不足声明 = 是”，看起来像没有回答完整；Q7 只检索到 1 个片段，答案信息量偏少；Q1 和 3.1 的掌握度问题都把 README 的“资料问答（RAG）”小节排进前几名，关键词分数高但内容与问题无关，说明检索排序仍有噪声。结论是：RAG 的作用是把回答限制在资料范围内、并提供可核对的出处，不能代替人工判断内容是否准确；本次样例中 RAG 在项目专属事实和“资料不足”处理上明显更好，直接模型在措辞自然度和通用知识上不差，但容易把通用知识当成项目事实。

3.6 模块调用与后续使用

检索和回答生成整理成可调用模块，依赖关系和失败返回见表 5-10。知识库准备方法：把资料登记到 `backend/kb/sources.json`，执行 `python3 -m app.tools.build_kb` 生成 `chunks.jsonl` 即可；资料更新后重新执行同一条命令。

表 5-10: 模块、输入输出与失败返回

模块	依赖	输入与输出	失败返回
`qa_service.answer_question`	retriever、prompts、llm_client	问题、模式、Top-k、Prompt 版本 → 回答、引用、检索片段、实际上下文、耗时、tokens、费用	空问题 400、问题过长 400、知识库未构建 400
`retriever.search`	kb_service	问题、Top-k、阈值 → 按分数排序的片段	无结果时返回空列表
`kb_service.build_index`	sources.json	资料清单 → chunks.jsonl 与索引元数据	资料文件缺失 500
`llm_client.chat`	httpx	messages、模型参数 → 内容、用量、耗时	未配置 503、连接失败 503、超时 504、鉴权失败 502、限流 503、返回结构异常 502
`llm_client.chat_json`	Pydantic	messages、输出结构 → 校验后的对象	两次都不合法 502

入口有页面、接口和命令行三种：页面是 `/api/qa/ask` 对应的“资料问答”页（RAG/直接模型可切换，显示回答、出处、检索片段、耗时和费用）；接口为 `POST /api/qa/ask`、`GET /api/qa/kb`、`GET /api/qa/kb/chunks`、`POST /api/qa/kb/rebuild`；命令行包括 `ask_cli`（问答，可打印检索结果和完整输入）、`build_kb`、`inspect_chunks`、`compare_test`、`context_check`、`fail_demo`、`parse_task`。另外有 10 个离线单元测试，和实验 4 的 10 个用例一起共 20 个，本次全部通过。



图 5-18: 页面入口：直接模型模式（不给资料）的回答



图 5-19: 页面入口：超出范围的问题返回资料不足

```
$ cd backend && python3 -m pytest tests -q --no-header --color=yes 2>/dev/null
....................                                                    [100%]
20 passed in 0.48s

$
```

图 5-20: 单元测试：20 个用例全部通过

```
模型调用失败： 无法连接模型服务（ http://127.0.0.1:9），请检查网络或服务地址（ LLM_CONNECTI
ON_ERROR）
2026-10-26 10:00:44 INFO [studypilot] 调用模型： model=deepseek-chat， messages=2
条， 输入约 186 字符
2026-10-26 10:00:54 INFO [studypilot] 模型返回： 耗时 10479 ms， tokens=153
2026-10-26 10:00:54 INFO [studypilot] 调用模型： model=deepseek-chat， messages=2
条， 输入约 171 字符
2026-10-26 10:00:56 INFO [studypilot] 模型返回： 耗时 1552 ms， tokens=142
2026-10-26 10:00:56 INFO [studypilot] 调用模型： model=deepseek-chat， messages=4
条， 输入约 496 字符
2026-10-26 10:00:57 INFO [studypilot] 模型返回： 耗时 881 ms， tokens=244
2026-10-26 10:01:01 INFO [studypilot] 调用模型： model=deepseek-chat， messages=1
条， 输入约 1484 字符
2026-10-26 10:01:02 INFO [studypilot] 模型返回： 耗时 1252 ms， tokens=767
2026-10-26 10:01:02 INFO [studypilot] 调用模型： model=deepseek-chat， messages=1
条， 输入约 1116 字符
2026-10-26 10:01:03 INFO [studypilot] 模型返回： 耗时 1236 ms， tokens=728
2026-10-26 10:01:04 WARNING [studypilot] 检索无结果， 未调用模型： 帮我写一首关于春天的诗
2026-10-26 10:01:07 INFO [studypilot] 调用模型： model=deepseek-chat， messages=1
条， 输入约 1484 字符
2026-10-26 10:01:08 INFO [studypilot] 模型返回： 耗时 1199 ms， tokens=765

$
```

图 5-21: 运行日志：调用失败分类、模型调用/返回用量与检索无结果

结论是继续在 StudyPilot 中使用 RAG，作为“资料问答”功能保留，但只作为参考，不直接生成或修改学习计划。当前局限：一是关键词检索对用词差异敏感，同义词表靠

手工维护；二是中文按二字切分会产生无意义组合，可能误匹配；三是知识库只有 5 篇项目文档，没有教材内容，回答范围有限；四是引用核对只看词语重合度，不能完全判断语义是否支持回答（本次 Q6 的 0.098 弱引用就是例子）；五是 RAG 的 tokens 和费用约为直接模型的 3.3 倍和 2.3 倍；六是资料清洗还不够彻底，代码围栏和表格列宽串仍会进入检索。下一步计划：把 ER 图等图片内容补充成文字资料，加入课程复习资料；把 8 个固定问题作为回归测试集，修改 Prompt 或检索参数后自动跑一遍对比；改进上下文过滤，对“资料问答（RAG）”这类命令说明片段做来源降权；完善 LaTeX/Markdown 清洗规则，去掉代码围栏和表格列宽串；后续再考虑关键词与向量检索结合。

3.7 关键代码

模型调用与失败分类的代码见代码 5-1，RAG 主流程见代码 5-2；代码直接来自项目文件 backend/app/service/ 下的实现，省略了与本实验无关的辅助函数。

```python
def chat(
    messages: list[dict],
    *,
    model: str | None = None,
    temperature: float = 0.2,
    max_tokens: int = 900,
    timeout: float | None = None,
    json_mode: bool = False,
    base_url: str | None = None,
    api_key_override: str | None = None,
) -> dict:
    """调用模型并返回内容、用量和耗时；失败时抛出带错误码的业务异常。"""
    key = api_key_override if api_key_override is not None else api_key()
    if not key:
        raise BusinessError("未配置 DEEPSEEK_API_KEY，无法调用模型服务", 503, "LLM_NOT_CONFIGURED")

    url = f"{(base_url or DEFAULT_BASE_URL).rstrip('/')}/chat/completions"
    payload = {
        "model": model or DEFAULT_MODEL,
        "messages": messages,
        "temperature": temperature,
        "max_tokens": max_tokens,
        "stream": False,
    }
    if json_mode:
        payload["response_format"] = {"type": "json_object"}

    log.info("调用模型：model=%s，messages=%d 条，输入约 %d 字符", payload["model"], len(messages),
             sum(len(m.get("content", "")) for m in messages))
    started = time.time()
    try:
        resp = httpx.post(
            url,
            json=payload,
            headers={"Authorization": f"Bearer {key}", "Content-Type": "application/json"},
            timeout=timeout or DEFAULT_TIMEOUT,
        )
        resp.raise_for_status()
        data = resp.json()
    except Exception as exc:  # noqa: BLE001 - 统一分类后抛出业务异常
        error = _classify(exc, url.rsplit("/chat/completions", 1)[0])
        log.error("模型调用失败：%s（%s）", error.message, error.code)
        raise error from exc

    latency_ms = int((time.time() - started) * 1000)
    try:
        content = data["choices"][0]["message"]["content"]
    except (KeyError, IndexError) as exc:
        log.error("模型返回结构异常：%s", str(data)[:200])
        raise BusinessError("模型返回内容结构异常，无法解析", 502, "LLM_BAD_RESPONSE") from exc

```

```
    result = {
        "content": content,
        "model": data.get("model", payload["model"]),
        "usage": data.get("usage", {}),
        "latency_ms": latency_ms,
        "finish_reason": data["choices"][0].get("finish_reason"),
    }
    log.info("模型返回：耗时 %d ms，tokens=%s", latency_ms, result["usage"].get("total_tokens"))
    return result
```

代码 5-1: 模型调用与失败分类（backend/app/service/llm_client.py 第 49—108 行）

```
    hits: list[dict] = []
    context = ""
    if mode == "rag":
        hits = retriever.search(question, top_k=top_k)
        if not hits:
            return _empty_retrieval(question, mode, top_k, prompt_version)
        context = build_context(hits)

    started = time.time()
    if mode == "rag" and prompt_version in {"v2", "v3"}:
        messages = prompts.rag_messages(question, context, prompt_version)
        parsed, raw = llm_client.chat_json(messages, prompts.RagAnswer, temperature=temperature,
     max_tokens=700)
        answer, citations, sufficient = parsed.answer, sorted(set(parsed.citations)), parsed.sufficient
    else:
        messages = (
            prompts.rag_messages(question, context, "v1")
            if mode == "rag"
            else prompts.direct_messages(question, prompt_version)
        )
        raw = llm_client.chat(messages, temperature=temperature, max_tokens=700)
        answer = raw["content"].strip()
        citations = sorted({int(n) for n in re.findall(r"\[(\d+)\]", answer)})
        sufficient = "资料不足" not in answer

    result = {
        "question": question,
        "mode": mode,
        "prompt_version": prompt_version,
        "answer": answer,
        "citations": [c for c in citations if 1 <= c <= len(hits)] if mode == "rag" else [],
        "sufficient": sufficient,
        "retrieved": [
            {
                "index": i,
                "chunk_id": hit["chunk_id"],
                "source_title": hit["source_title"],
                "section": hit["section"],
                "path": hit["path"],
                "score": hit["score"],
                "chars": hit["chars"],
                "preview": hit["text"][:120] + ("…" if hit["chars"] > 120 else ""),
            }
            for i, hit in enumerate(hits, start=1)
        ],
```

```
        "citation_checks": verify_citations(answer, hits) if mode == "rag" else [],
        "context": context,
        "usage": raw["usage"],
        "latency_ms": raw["latency_ms"],
        "model": raw["model"],
        "cost_yuan": estimate_cost(raw["usage"]),
        "context_chars": len(context),
        "total_ms": int((time.time() - started) * 1000),
    }
    log.info(
        "问答完成: mode=%s, prompt=%s, 检索 %d 个片段, 耗时 %d ms, tokens=%s, 引用 %s",
        mode, prompt_version, len(hits), result["latency_ms"], result["usage"].get("total_tokens"),
     result["citations"],
    )
    return result
```

代码 5-2: RAG 主流程: 检索、上下文、调用与引用校验（backend/app/service/qa_service.py 第 99—156 行）

3.8 版本记录

代码提交到 GitHub 仓库 https://github.com/luohao711/StudyPilot，本实验的 RAG 实现对应 commit 7fe795a、tag v0.3，后续提交只补充报告、截图与重跑证据；报告中的重跑数据与截图来自本机 2026-10-26 的运行，源文件、截图和运行证据放在 reports/experiment5/ 与 backend/tests/data/ 下。

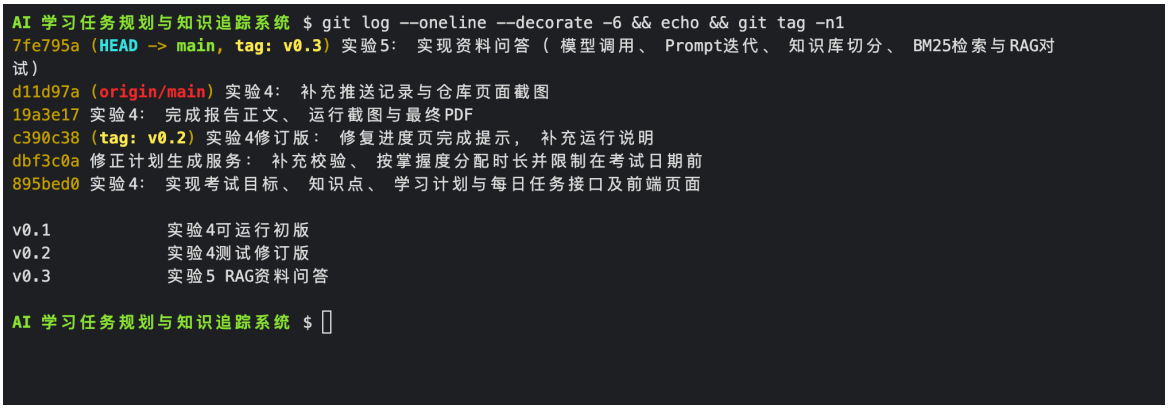


图 5-22: 版本记录：本实验对应的 Commit 与 Tag

```
 '--- 第' $i '次失败， 重试 ---'; sleep 10; done
fatal: unable to access 'https://github.com/luohao711/StudyPilot.git/': Failed to connect to github.com port 443 after
 75003 ms: Couldn't connect to server
--- 第 1 次失败， 重试 ---
Enumerating objects: 97, done.
Counting objects: 100% (97/97), done.
Delta compression using up to 8 threads
Compressing objects: 100% (73/73), done.
Writing objects: 100% (77/77), 10.94 MiB | 2.88 MiB/s, done.
Total 77 (delta 13), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (13/13), completed with 12 local objects.
To https://github.com/luohao711/StudyPilot.git
   d11d97a..7fe795a  main -> main
 * [new tag]         v0.3 -> v0.3

AI 学习任务规划与知识追踪系统 $
```

图 5-23: 推送到 GitHub 的记录

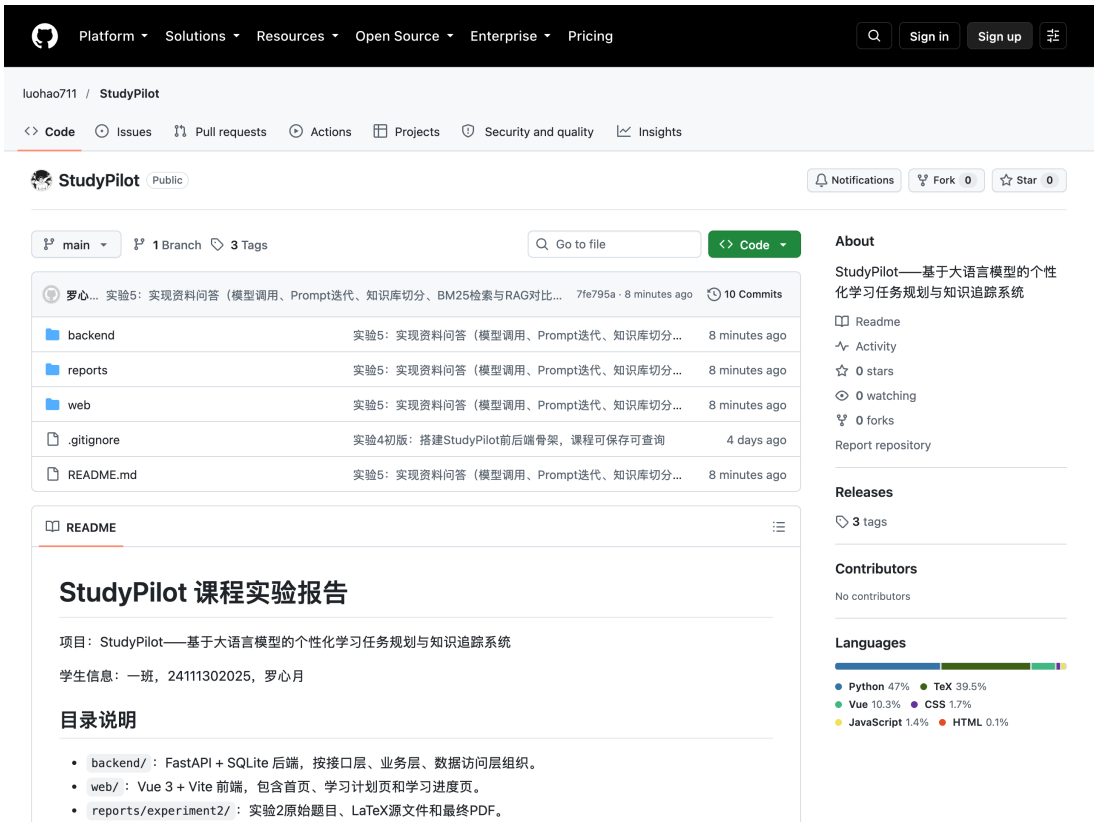


图 5-24: StudyPilot 仓库页面

4. 实验小结

本次完成了 StudyPilot 的资料问答功能：用 DeepSeek 的 chat completions 接口做模型调用，把 5 篇项目文档转成 77 个片段的知识库，用 BM25 关键词检索做检索，再让模型依据检索到的片段生成带出处的回答，前端加了一个“资料问答”页面，命令行和接口也都能调用。

过程中主要做了四件事。第一是 Prompt 改了两次：初版没有格式要求，回答发散、使用 Markdown 列表且引用格式不受约束；第二版要求只依据资料、限制长度并输出 JSON，引用可以逐条核对；测试三组输入时又发现“帮我设置考试目标”这类缺少参数的操作请求会被当成普通问题回答，于是加了追问规则。对比 v2 和 v3 可见 v3 会先说明缺少参数、再列需要补充的信息，但回答里仍带出字段名和接口，追问还不够简洁。第二是知识库重建：抽查时问“数据库保存了哪些数据”，旧版只命中实验 3 的设计说明；补一份数据模型说明后回答正确；本次重跑还把 README 的资料问答说明纳入知识库，片段数从 74 变为 77，说明资料更新后必须重建索引。第三是检索调整：加入标题加权和同义词扩展后，同一个问题的正确片段从检索不到变成排在第 1。第四是上下文工程：Top-k=6 时上下文 1577 字、包含 LaTeX 编译命令等无关内容和重复命令，按分数精简到 657 字（减少 58%），prompt tokens 从 1080 降到 541，回答和引用没有变化；但按分数精简不能去掉所有噪声，README 的高分无关片段仍会保留。

对比测试用 8 个固定问题跑了 16 次测试（实际 15 次模型调用），共 7316 tokens、约 0.0224 元、22.0 秒。直接模型在项目专属问题上会给出看似合理但不符实际的内容（架构分层名称、自动更新掌握度、编造 MVP 排除项），有检索结果且给出引用的 RAG 回答都能对应到原片段；Q8 未调用模型、没有引用，两个资料无答案的问题也都明确说明资料不足。RAG 也不是没有问题：Q1 已经答对却补充“资料不足”，Q7 检索片段偏少，README 的 RAG 说明会排进前几名，引用核对只能看词语重合度（本次有 1 处弱引用）。局限是关键词检索依赖用词、知识库范围有限、RAG 的 tokens 和费用约为直接模型的 3.3 倍和 2.3 倍；后续用固定的 8 个问题做回归测试，补充资料并改进过滤，再考虑关键词与向量检索结合。